Chess Blunder-Risk Analyzer: D1 Detailed Design

Braden Hayes, Nate Poppe

We are still working through several details with our advisor, including the training label, training data, model choice, and hosting. This document reflects our current plan and may change after that meeting.

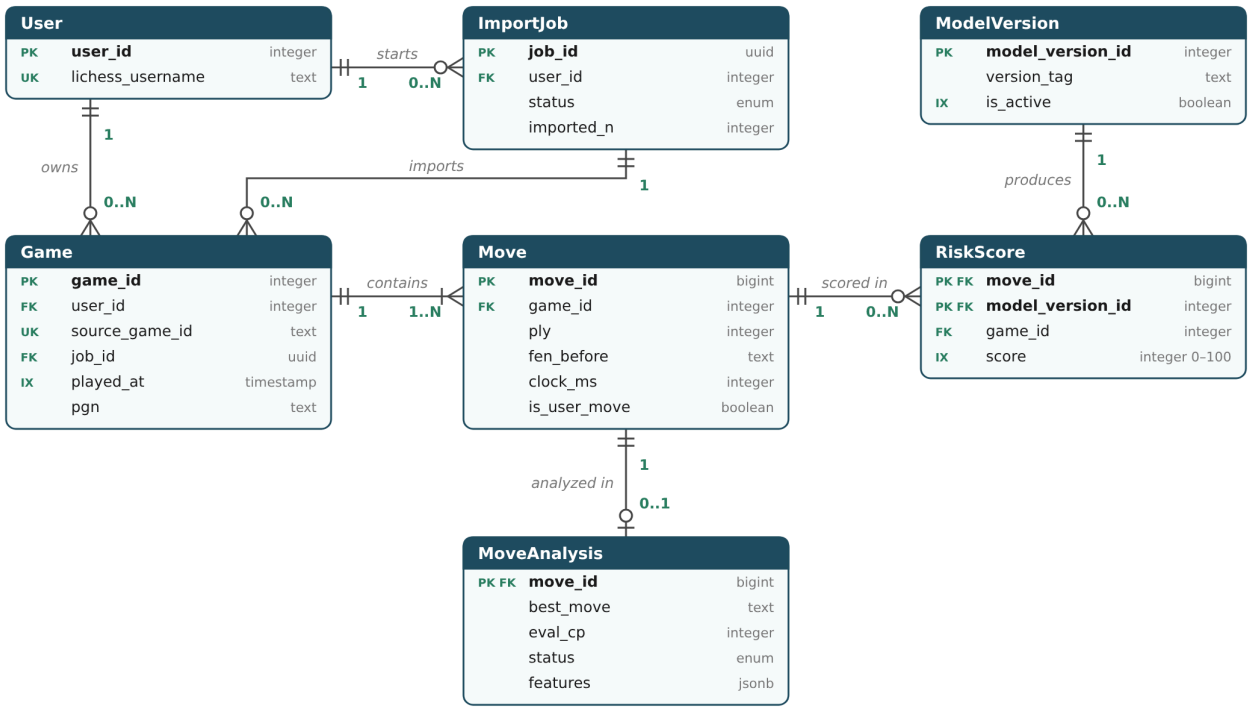
Header, scope, and conventions

Goal statement. The goal of the Chess Blunder-Risk Analyzer is to develop a machine-learning-based system that estimates the likelihood that a chess player will make a blunder from a given position. The system will analyze completed chess games using factors such as player rating, clock time, game phase, position characteristics, and engine-derived information. It will then present move-level risk scores and broader patterns that help players identify situations in which they are more likely to make mistakes.

Scope. This document covers three D0 components: Game and Results Storage, Game Import and Validation, and Chess Feature Extraction. The User Interface, Blunder-Risk Model, and Risk Explanation and Analytics will be covered in D2.

Reading the data model diagram. Each box is one table in our database. The dark header is the table name, and each row below it is a column, with its type on the right. Only columns that affect how the system behaves are shown. The green tags on the left mark keys: PK is a primary key, FK is a foreign key that points to another table, UK means the column is part of a unique rule, and IX means the column is indexed. Each line connects two related tables and is labeled with how they relate. The symbols at each end of a line, repeated as green text, show how many rows can be on that side: 1 means exactly one, 0..1 means zero or one, 0..N means zero or more, and 1..N means one or more.

Data model



We chose a relational database (PostgreSQL). Our data has a clear shape: users have games, games have moves, and moves have scores. A relational database also lets us enforce our most important rules directly: the same game can't be saved twice for one user, every score has to belong to a real model version, and a game is either saved completely or not at all.

Why each table exists

Table	Why it is its own table
Move	Replay, engine analysis, and risk scores all need to point at one specific move, which is hard if moves are stored as a list inside a game.
ModelVersion	Storing versions in their own table means every score is tied to a version that actually exists, instead of a free-text label.
ImportJob	Keeps track of each import's progress, so an import that gets interrupted can pick up where it stopped.
MoveAnalysis	Engine results are slow to produce. Keeping them separate means a new model can rescore old games without running the engine again.

How the tables relate

Relationship	Type	Why
User to Game	One-to-many	If two users import the same game, each keeps their own copy, so one account's data never mixes with another's.

Relationship	Type	Why
Move to ModelVersion	Many-to-many, through RiskScore	A move can be scored by several model versions, which lets us compare versions on the exact same games.
Move to MoveAnalysis	One to zero-or-one	A move with only one legal option, or one the engine failed on, has no analysis.

Indexes

Index	Purpose
Game: user_id + source_game_id (unique)	Stops the same game from being imported twice for a user.
Game: user_id + played_at	Lists a user's games newest first, and tells an interrupted import where to resume.
RiskScore: game_id + model_version_id + score	Finds the riskiest moves in a game without sorting every move.
ModelVersion: is_active (unique when true)	Guarantees only one model version is active at a time.

Core algorithms

There are two pieces of the system that would break the product if they were slow or wrong: the game import, and the engine analysis that runs on every move.

A1: Resumable game import

Purpose. Download a player's recent finished games and save them. If the import is interrupted partway through, the games already saved are kept, and resuming does not download them again. Games that are still being played are never saved.

Inputs. - The player's Lichess username - The time control (bullet, blitz, rapid, or classical) - How many recent games to import, or an uploaded PGN file in place of a username

Outputs. - The import's status: running, interrupted, or done - How many games have been saved - The saved games and their moves

Approach. Lichess sends the games newest first. Each game is checked to make sure it is finished, is standard chess, and has at least two moves. The game and all its moves are then saved together in one step, so a game is never half saved. If the import is interrupted, it is marked as interrupted. When the player resumes, we only ask Lichess for games older than the oldest one already saved.

Complexity. O(g × m), where g is the number of games and m is the number of moves per game. The work grows in a straight line. - At a realistic size of 50 games with about 80 moves each, that is about 4,000 moves to check and save, which takes well under a second. Most of the time is spent waiting on the download. - At 100 times that size, about 400,000

moves, it still grows in a straight line. This doesn't matter in practice because imports are capped at 50 games.

Alternatives we considered. - Re-downloading everything on resume is simpler, but it wastes time and repeats work we already did. - Saving the whole import in one step would lose every game if anything went wrong partway through. - Getting a list of game IDs first and then fetching each game one by one would send far more requests to Lichess.

Edge cases.

Situation	What happens
The username doesn't exist	The import is turned down before it starts.
The player has no matching games	The import finishes with nothing saved.
A game was already saved	It is skipped.
A game is still being played	It is skipped and never analyzed.
The import gets interrupted	The player is told, and can resume later.
One game in an uploaded PGN file is broken	That game is skipped and the rest are imported.
The player presses resume twice	The second request is turned down.

A2: Engine analysis

Purpose. Run each of the player's moves through Stockfish and turn the result into the information the model needs. It has to finish quickly enough that the player isn't left waiting, and running the same game twice has to give the same result, so different model versions can be compared fairly.

Inputs. - The board position before each of the player's moves - The time left on the clock at each move - How deeply the engine should search - How many engines to run at once

Outputs. - The engine's best move for each position - The engine's evaluation of each position - Whether the analysis succeeded, was skipped, or failed - The features passed to the model

Approach. Several copies of Stockfish run side by side, one per spare CPU core. Each one searches a position to the same fixed depth, so the results don't change from run to run. If a position shows up more than once, the earlier result is reused. A move with only one legal option skips the engine entirely. If the engine crashes or takes too long, the position is tried once more and then marked as failed.

Complexity. O(p × d ÷ e), where p is the number of positions, d is how deep the engine searches, and e is how many engines run at once. Everything other than the engine itself is fast by comparison. - At a realistic size of 50 games with about 40 player moves each, that is about 2,000 positions. With 4 engines running at once, each handles about 500. - At 100 times that size, about 200,000 positions would take far too long to wait for. That only matters for building a training set, which won't be done this way. Normal use only looks at a player's recent games.

Alternatives we considered. - Giving the engine a fixed amount of time per move would give different results depending on how busy the computer is. - Searching to a fixed depth in moves, instead of a fixed amount of work, can take wildly different amounts of time on complicated positions. - Analyzing the opponent's moves as well would double the work for no benefit, since only the player's moves are scored.

Edge cases.

Situation	What happens
A game has no moves by the player	There is nothing to analyze.
A move had only one legal option	It isn't scored, since there was no decision to get wrong.
The engine crashes or freezes	The position is tried once more, then marked as failed.
Two moves tie for third-riskiest	The earlier move is shown.
A game has fewer than three scored moves	Only those moves are shown.
Stockfish isn't running	New imports are turned down until it is back.

Build-versus-reuse decisions

We reuse anything that is already well built and focus our own time on what is specific to this project. Each library below was checked for how mature and actively maintained it is, its license, whether it is fast enough for our use, and how well it fits with Python.

What we reuse

Library or service	What we use it for	License	Why
Lichess API	Downloading a player's games	Free (Lichess terms)	It is where the games and clock times come from.
Authlib, with Lichess sign-in	Signing users in	BSD-3-Clause	Authentication is easy to get wrong, so we don't build our own.
python-chess	Chess rules, reading game files, board positions	GPL-3.0	Mature and widely used. Rewriting chess rules is out of scope.
Stockfish	Analyzing positions	GPL-3.0	The strongest free chess engine.
FastAPI and Pydantic	The web API and checking user input	MIT	Input checking and API docs come from one definition.
SQLAlchemy and Alembic	Talking to the database and changing its structure over time	MIT	Mature and widely used with PostgreSQL.

Library or service	What we use it for	License	Why
Python standard library and PostgreSQL	Handling dates and sorting results	PSF / PostgreSQL	Already built in, so there is nothing to write.

What we build

Piece	Why we build it
Import and resume logic (A1)	It depends on exactly how our imports work.
Running several engines side by side (A2)	A small amount of code using Python's built-in asyncio, with no extra service needed.
Turning engine analysis into model features	This is the core of our project.

API contract

All endpoints start with `/v1`. The user is identified by their sign-in session.

Start an import from Lichess

`POST /v1/imports`

- **Sends:** a username (required, 2 to 30 characters), a time control (required: bullet, blitz, rapid, or classical), and the number of games (required, 1 to 50).
- **Returns:** the import's ID, its status, and how many games have been saved so far.
- **Fails when:** a field is missing or invalid, the user isn't signed in, the username doesn't exist, the user already has an import running, or the chess engine isn't running.

Start an import from a PGN file

`POST /v1/imports/pgn`

- **Sends:** a .pgn file (required).
- **Returns:** the import's ID, its status, how many games were read, and how many were skipped.
- **Fails when:** the file isn't a PGN, the file is too large, or none of the games in it can be used.

Check an import's progress

`GET /v1/imports/{id}`

- **Sends:** the import's ID (required).
- **Returns:** the import's status, how many games are saved, and how many have been analyzed.

- **Fails when:** the import doesn't exist or belongs to another user.

Resume an interrupted import

`POST /v1/imports/{id}/resume`

- **Sends:** the import's ID (required).
- **Returns:** the import's ID and its new status (running).
- **Fails when:** the import doesn't exist, or it isn't currently interrupted.

Get a game's moves and risk scores

`GET /v1/games/{id}/moves`

- **Sends:** the game's ID (required), and a model version (optional, defaults to the current one).
- **Returns:** for each move, its number, the board position, the time on the clock, its risk score (0 to 100, or empty if it wasn't scored), and the engine's best move.
- **Fails when:** the game or the model version doesn't exist.

Example

Request:

```
POST /v1/imports

{"username": "example_player", "perf_type": "blitz", "max_games": 20}
```

Response:

```
{"job_id": "3f9c2a1e-8b4d-4f6a-9e21-5c7d0b8a4e12", "status": "running", "imported_n": 0}
```

Versioning

The API version is the `/v1` at the start of every path. Removing or renaming a field, changing what a field means, or adding a new required field counts as a breaking change and would go under `/v2`. Adding optional fields or new endpoints does not.